MARATONA DE PROGRAMAÇÃO 1

Apostila de Revisão

Material personalizado construído a partir da análise de todos os seus códigos
(Listas 01, 02, 03, Simulado 0, Aulas e Estudos)

Revisão · referência rápida · guia de melhoria para os simulados

Jefferson Vinicius — Sistemas de Informação, UNIFEI

Sumário

1. O que já foi estudado
2. Sua evolução entre as listas
3. Estruturas de Dados
4. STL em C++
5. O que eu deveria ter percebido?
6. Padrões de problemas que preciso reconhecer
7. Exemplos práticos de melhoria
8. Complexidade e análise de algoritmos
9. Grafos
10. STL aplicada a Grafos
11. Checklist para os simulados
12. Exercícios de revisão
13. Mapa mental de reconhecimento
14. Próximos pontos a estudar

---

1. O que já foi estudado

Conteúdos cobertos

Fonte	Tópicos
Lista 01 (15 problemas)	Matemática básica, geometria (distância entre pontos), loops, condicionais, fatorial, divisão, contagem, bounding box, lógica de xadrez (dama), impedimento em futebol
Lista 02 (16 problemas)	Strings com `char[]`, manipulação de caracteres (LEDs, criptografia), palíndromo, simulação (fila, sub-prime), ordenação manual (Bubble Sort), soma de intervalos, XOR bit-a-bit, pontos dentro de retângulo
Lista 03 (16 problemas)	STL (`stack`, `queue`, `map`, `set`, `sort`), `string` do C++, Josephus com `queue`, parênteses com `stack`, contagem de frequência com `map`, elementos distintos com `set`, eleição com `map`, ordenação com `sort`
Simulado 0 (5 problemas)	Simulação direta, troca de vetores, contagem condicional, soma de dígitos
Aulas	Lista encadeada manual, introdução a `vector`, introdução a Grafos (teoria de DFS/BFS)

Linguagem e Estilo

- **Lista 01:** 100% C puro (`scanf`, `printf`, `char[]`, sem `using namespace std`).
- **Lista 02:** Predominantemente C puro, com pequenas incursões em C++ (`bits/stdc++.h`), mas usando `scanf`/`printf` e arrays de `char`. **Ordenação feita manualmente com Bubble Sort**.
- **Lista 03: Virada para C++ competitivo!** Uso de `cin`/`cout`, `using namespace std`, e STL real: `stack`, `queue`, `map`, `set`, `sort`, `string`, `getline`.

> **IMPORTANTE**
> A Lista 03 marca a virada mais importante do seu aprendizado: você deixou de programar "C com extensão `.cpp`" e começou a usar C++ de verdade.

---

2. Sua evolução entre as listas

Lista 01 → Lista 02: Aprofundamento em C

Você consolidou a base de **loops**, **condicionais** e **vetores nativos**. Começou a lidar com: - Strings como arrays de `char` (problemas 1241, 1257, 3301) - Entrada com múltiplos casos de teste (`while(scanf(...) != EOF)`) - Manipulação de

caracteres via tabela ASCII (`c - '0'` , `c - 'A'`)

Problema: Você implementou **Bubble Sort manualmente** nos problemas 1192 e 1457, gastando ~15 linhas para algo que `sort()` resolve em 1.

Lista 02 → Lista 03: Salto para C++ e STL

Na Lista 03, você demonstrou que aprendeu a usar: - `stack<char>` para casamento de parênteses (1068, 1449) - `queue<int>` para simulação FIFO e Josephus (1030, 1555) - `map<string, int>` para contagem de frequência (1769, 1863, 2286) - `set<int>` para elementos distintos (1962) - `sort()` com arrays (1259) - `string::substr()` para comparação de sufixos (1241 refeito)

> **DICA**
> **Comparação direta da sua evolução:** O problema **1241 (Encaixa ou Não)** aparece na Lista 02 e na Lista 03. Na Lista 02, você fez 40 linhas em C com loops manuais para comparar sufixos. Na Lista 03, você resolveu em 20 linhas usando `string::substr()` . **Essa é exatamente a melhoria que precisamos consolidar.**

---

3. Estruturas de Dados

3.1 Vetores / Arrays

O QUE É?

Sequência contígua de posições na memória, acessadas por índice.

QUANDO RECONHECER?

- “Leia N elementos” → Vetor.
- Acesso direto por posição (índice) é necessário.
- Precisa ordenar os dados.

COMO APARECE NOS SEUS CÓDIGOS

Praticamente **todos os seus problemas** usam vetores: desde o simples `int vet[n]` até o `float notas[n]` .

ARMADILHAS QUE VOCÊ JÁ COMETEU

1. Declarar vetor com tamanho de variável não inicializada:

```
// ✗ Seu código no 1030.cpp (Lista 02) — 'pessoas' ainda é lixo!
int nc, pessoas, casa;
int vet[pessoas] = {0};
```

2. VLA (Variable-Length Array) — Funciona mas é perigoso:

```
// Funciona no GCC, mas NÃO é padrão C++
int n;
cin >> n;
int vet[n]; // VLA — pode estourar a stack se n for grande
```

Solução segura:

```
int n;
cin >> n;
vector<int> vet(n); // ✓ Aloca no heap, seguro para qualquer tamanho
```

REGRA DE OURO

> Se o tamanho é conhecido em tempo de compilação e pequeno (ex: `int vet[26]` para letras do alfabeto) → use array nativo. Se o tamanho depende da entrada → use `vector<int>` .

3.2 Pilha (Stack)

O QUE É?

Coleção LIFO (Last In, First Out): o último a entrar é o primeiro a sair.

PARA QUE SERVE?

Resolver problemas de **casamento/emparelhamento**: parênteses, tags HTML, operações de desfazer.

OPERAÇÕES E COMPLEXIDADE

Operação	Método	Complexidade
Inserir no topo	`push(x)`	$O(1)$
Remover do topo	`pop()`	$O(1)$
Ver o topo	`top()`	$O(1)$
Está vazia?	`empty()`	$O(1)$
Tamanho	`size()`	$O(1)$

COMO APARECE NOS SEUS CÓDIGOS

1068 — Parênteses Válidos (Lista 03):

```
stack<char> pilha;
for (char c : s) {
    if (c == '(') pilha.push(c);
    else if (c == ')') {
        if (pilha.empty()) { valida = false; break; }
        pilha.pop();
    }
}
if (valida && pilha.empty()) cout << "correct\n";
```

✓ Excelente uso! Solução limpa e correta.

1449 — Fila do Reajuste (Lista 03):

```
stack<int> pilha;
// v == 1 → push, v == 2 → pop
```

✓ Também correto. Você aplicou a pilha para simular entrada/saída.

COMO RECONHECER EM UMA QUESTÃO DE PROVA

> - "Verificar se parênteses/colchetes/chaves estão balanceados" → **`stack`**.
> - "Desfazer a última operação" (Ctrl+Z) → **`stack`**.
> - "Processar de trás para frente" → **`stack`**.
> - "Empilhar/desempilhar" → **`stack`**.

ERROS COMUNS

- Chamar `top()` ou `pop()` em uma pilha vazia → **comportamento indefinido** (crash).
- **Sempre** verifique `!pilha.empty()` antes de acessar.

3.3 Fila (Queue)

O QUE É?

Coleção FIFO (First In, First Out): o primeiro a entrar é o primeiro a sair. Como uma fila de banco.

OPERAÇÕES E COMPLEXIDADE

Operação	Método	Complexidade
Inserir no final	`push(x)`	$O(1)$
Remover da frente	`pop()`	$O(1)$
Ver o primeiro	`front()`	$O(1)$
Ver o último	`back()`	$O(1)$
Está vazia?	`empty()`	$O(1)$

COMO APARECE NOS SEUS CÓDIGOS

1030 — Josephus (Lista 03): A joia da sua evolução! Na Lista 02, você tentou simular com vetores e deu errado. Na Lista 03, resolveu com `queue` em poucas linhas:

```
queue<int> q;
for (int j = 1; j <= n; j++) q.push(j);

while (q.size() > 1) {
    for (int j = 1; j < k; j++) {
        q.push(q.front()); // Manda para o fim da fila
        q.pop();           // Remove da frente
    }
    q.pop(); // Elimina o k-ésimo
}
cout << q.front(); // Sobrevivente
```

1555 — Fila do Supermercado (Lista 03):

```
queue<int> fila;
for (i = 0; i < n; i++) { cin >> e; fila.push(e); }
cin >> s;
for (i = 0; i < s; i++) fila.pop(); // Remove s pessoas da frente
cout << fila.front() << "\n";       // Quem é o próximo?
```

COMO RECONHECER EM UMA QUESTÃO DE PROVA

> - "Fila de atendimento / ordem de chegada" → **`queue`**.
> - "BFS (Busca em Largura)" → **`queue`**.
> - "Simulação circular" (quem sai da frente vai pro final) → **`queue`**.
> - Problema do Josephus → **`queue`**.

3.4 Map (Dicionário Ordenado)

O QUE É?

Container associativo que mapeia uma **chave** para um **valor**. Internamente é uma árvore rubro-negra auto-balanceada. As chaves ficam **sempre ordenadas**.

OPERAÇÕES E COMPLEXIDADE

Operação	Sintaxe	Complexidade
Inserir/atualizar	`m[chave] = valor`	$O(\log N)$
Acessar	`m[chave]`	$O(\log N)$
Verificar existência	`m.count(chave)` ou `m.find(chave)`	$O(\log N)$
Iterar (em ordem)	`for (auto p : m)`	$O(N)$

COMO APARECE NOS SEUS CÓDIGOS

1769 — Roupa (Lista 03) — Combinatória com frequência:

```
map<string, int> freq;
for (int i = 0; i < l; i++) {
    string peca; cin >> peca;
    freq[peca]++;  // Conta quantas peças de cada tipo
}
int result = 1;
for (auto p : freq) result *= (p.second + 1);
result--;
cout << result << "\n";
```

✓ Uso perfeito de `map` para agrupar dados por categoria.

1863 — Frequência de Palavras (Lista 03):

```
map<string, int> freq;
for (i = 0; i < n; i++) { string s; cin >> s; freq[s]++; }
for (auto p : freq) cout << p.first << " " << p.second << "\n";
```

✓ Correto. Como o `map` mantém as chaves em **ordem alfabética**, a saída já sai ordenada sem precisar de `sort`.

2286 — Eleição (Lista 03):

```
map<string, int> votos;
// ... conta votos ...
// ... encontra o vencedor e verifica empate ...
```

✓ Uso correto. Você até tratou o caso de empate — boa maturidade!

COMO RECONHECER EM UMA QUESTÃO DE PROVA

> - "Contar frequência de elementos" (palavras, nomes, números) → **`map`**.
> - "Associar uma chave a um valor" (nome → nota, cidade → população) → **`map`**.
> - "Preciso que a saída esteja em ordem alfabética/numérica das chaves" → **`map`** (já sai ordenado).
> - Se a **ordem das chaves não importa** e você quer mais velocidade → **`unordered_map`** ($O(1)$ amortizado).

ERROS COMUNS

- `m[chave]` cria a chave com valor 0 se ela não existir! Se você apenas quer verificar existência, use `m.count(chave)` ou `m.find(chave)`.

3.5 Set (Conjunto Ordenado)

O QUE É?

Conjunto que **não permite duplicatas** e mantém os elementos **sempre ordenados**.

OPERAÇÕES E COMPLEXIDADE

Operação	Sintaxe	Complexidade
Inserir	`s.insert(x)`	$O(\log N)$
Remover	`s.erase(x)`	$O(\log N)$
Verificar existência	`s.count(x)`	$O(\log N)$
Tamanho (= elementos únicos)	`s.size()`	$O(1)$

COMO APARECE NOS SEUS CÓDIGOS

1962 — Elementos Distintos (Lista 03):

```cpp
set<int> s;
for (int i = 0; i < n; i++) {
    int v; cin >> v;
    s.insert(v); // Duplicatas são ignoradas automaticamente
}
cout << s.size() << "\n"; // Quantidade de elementos únicos
```

✓ Uso perfeito! Solução mínima e elegante.

COMO RECONHECER EM UMA QUESTÃO DE PROVA

> - "Quantos elementos **distintos/diferentes/únicos**?" → `set`.
> - "Remover duplicatas" → `set`.
> - "Verificar se um elemento já apareceu antes" → `set`.

3.6 String do C++ vs char[] do C

Característica	`char s[100]` (C)	`string s` (C++)
Tamanho	Fixo, você define	Cresce automaticamente
Comparação	`strcmp(a, b) == 0`	`a == b`
Concatenação	`strcat(dest, src)`	`a + b`
Substring	Manual com loops	`a.substr(pos, len)`
Tamanho	`strlen(s)` — *O(N)*	`s.size()` — *O(1)*
Risco de overflow	**Sim**	**Não**

SUA EVOLUÇÃO DIRETA: PROBLEMA 1241 (ENCAIXA OU NÃO)

Lista 02 (C puro) — 40 linhas:

```cpp
char a[1000], b[1000];
int lena = 0, lenb = 0;
for (j = 0; a[j] != '\0'; j++) lena++;
for (j = 0; b[j] != '\0'; j++) lenb++;
// ... loop manual comparando caractere por caractere ...
```

Lista 03 (C++ com string) — 20 linhas:

```cpp
string a, b;
cin >> a >> b;
string sufixo = a.substr(a.size() - b.size());
if (sufixo == b) cout << "encaixa\n";
```

> **DICA**
> **Lição:** Toda vez que você estiver escrevendo loops para comparar ou manipular strings, pergunte-se: "Existe um método da `string` do C++ que já faz isso?"

---

4. STL em C++

Referência Rápida das STL que você já usou

STL	Você usou em	Para que serve
`vector`	estudos/estudo.cpp	Array dinâmico seguro
`stack`	1068, 1449	Casamento de parênteses, LIFO
`queue`	1030, 1555	Josephus, simulação FIFO, BFS
`map`	1769, 1863, 2286	Frequência, dicionário chave→valor
`set`	1962	Elementos distintos
`string`	Lista 03 inteira	Manipulação de texto
`sort()`	1259	Ordenação $O(N \log N)$

STL que você deveria ter usado mas não usou

STL	Problema onde fez falta	O que você fez no lugar
`sort()`	1192 (Lista 02)	Bubble Sort manual (~15 linhas)
`sort() com cmp`	1457 (Lista 02)	Bubble Sort manual com `strcpy` (~20 linhas)
`string`	1241 (Lista 02)	`char[]` com loops manuais
`string`	3301 (Lista 02)	`char[100001]` com `strlen`
`vector<int>`	1030 (Lista 02)	`int vet[pessoas]` com variável não inicializada
deque	1142 (Lista 02)	Array com shift manual para mover elemento ao início
Prefix Sum	1985 (Lista 02)	Soma de intervalo com loop a cada query

---

5. O que eu deveria ter percebido?

Esta seção mostra momentos concretos nos seus próprios códigos onde uma abordagem melhor era possível.

Caso 1: Bubble Sort manual → `sort()` (Problema 1192)

O que você fez:

```
// 15 linhas de Bubble Sort — O(N²)
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - 1 - i; j++) {
        if (vet[j] > vet[j + 1]) {
            int temp = vet[j];
            vet[j] = vet[j + 1];
            vet[j + 1] = temp;
        }
    }
}
```

O que deveria ter feito:

```
sort(vet, vet + n); // 1 linha — O(N log N)
```

O que aprender: Sempre que o problema pedir para ordenar, use `sort()`. **Nunca** implemente Bubble Sort em competição.

Caso 2: Ordenação com struct em C → `sort()` com `pair` (Problema 1457)

O que você fez (Lista 02) — ~25 linhas:

```
char vet[n][21]; int val[n];
// Bubble Sort trocando strings com strcpy
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - 1 - i; j++) {
        if (val[j] < val[j + 1]) {
            int t = val[j]; val[j] = val[j + 1]; val[j + 1] = t;
            char tmp[21]; strcpy(tmp, vet[j]); strcpy(vet[j], vet[j + 1]); strcpy(vet[j + 1], tmp);
        }
    }
}
```

O que deveria ter feito — 5 linhas:

```
vector<pair<int, string>> lista(n);
for (int i = 0; i < n; i++) cin >> lista[i].second >> lista[i].first;
sort(lista.rbegin(), lista.rend()); // Ordena por valor decrescente
for (auto p : lista) cout << p.second << " " << p.first << "\n";
```

O que aprender: Quando precisa ordenar dados compostos (nome + valor), use `pair` e `sort`. O `pair` já compara automaticamente pelo `.first` e depois pelo `.second`.

Caso 3: Soma de intervalo com loop → Prefix Sum (Problema 1985)

O que você fez:

```
// Para CADA query, soma do índice a até b — O(Q × N) no pior caso
while (q--) {
    int a, b;
    scanf("%d %d", &a, &b);
    int soma = 0;
    for (int i = a; i <= b; i++) soma += vet[i];
    printf("%d\n", soma);
}
```

O que deveria ter percebido: Se existem muitas queries sobre o mesmo vetor, **pré-processar** uma vez custa $O(N)$ e cada query fica $O(1)$:

```
// Prefix Sum — pré-processamento O(N)
vector<long long> prefix(n + 1, 0);
for (int i = 1; i <= n; i++) prefix[i] = prefix[i - 1] + vet[i];

// Cada query — O(1)
while (q--) {
    int a, b;
    cin >> a >> b;
    cout << prefix[b] - prefix[a - 1] << "\n";
}
```

O que aprender: > "Se o problema pede **várias somas de intervalos** sobre o mesmo vetor que não muda" → **Prefix Sum**.

Caso 4: Movimentação na fila com shift manual → `deque` (Problema 1142)

O que você fez:

```
// Para mover um elemento para a frente da fila, fez shift manual em O(N)
int temp = fila[pos];
for (int i = pos; i > 0; i--) fila[i] = fila[i - 1];
fila[0] = temp;
```

O que deveria ter percebido: `deque<int>` permite inserir/remover no início E no final em $O(1)$:

```
deque<int> fila;
// push_front, push_back, pop_front, pop_back — todos O(1)
```

Caso 5: Bug silencioso — `==` quando queria `=` (Problema 1301)

No seu arquivo `aula/1301.cpp`:

```
if (a[i] > 0) a[i] == 1; // ✗ Comparação! Deveria ser a[i] = 1;
```

Isso **compila sem erro** em C/C++ mas **não faz nada** — a comparação retorna `true` ou `false` e o resultado é descartado.

Regra: Se o juiz dá *Wrong Answer* e seu código "parece certo", procure `==` onde deveria ser `=`.

Caso 6: Palíndromo com `char[]` → `string` + `reverse` (Problema 3301)

O que você fez:

```
char s[100001];
int esq = 0, dir = n - 1;
while (esq < dir) {
    if (s[esq] != s[dir]) { palindromo = 0; break; }
    esq++; dir--;
}
```

✓ A lógica está correta e eficiente. Mas em competição, uma abordagem mais concisa:

```
string s;
cin >> s;
string rev = s;
reverse(rev.begin(), rev.end());
cout << (s == rev ? "S" : "N") << "\n";
```

Ambas são $O(N)$. A segunda é mais rápida de **digitar**, o que importa na competição.

---

6. Padrões de problemas que preciso reconhecer

Tabela de Reconhecimento Rápido

Se o problema diz...	Pense em...	STL/Técnica
“Ordenar os dados”	Ordenação	`sort()`
“Ordenar por 2 critérios”	Ordenação customizada	`sort() + pair ou cmp`
“Quantos elementos distintos?”	Conjunto sem repetição	`set<int>`
“Contar frequência de cada X”	Dicionário	`map<string/int, int>`
“Associar nome a número”	Mapeamento	`map<string, int>`
“Parênteses balanceados”	Casamento LIFO	`stack<char>`
“Desfazer última operação”	LIFO	`stack`
“Fila de atendimento / ordem”	FIFO	`queue<int>`
“Simulação circular” (Josephus)	Fila circular	`queue<int>`
“Soma de intervalo” (múltiplas queries)	Pré-processamento	**Prefix Sum**
“Menor caminho / conexão”	Grafos	BFS / DFS
“Componentes conexas”	Grafos	DFS / BFS
“Relações entre pares de entidades”	Grafos	Lista de adjacência
“Encontrar o máximo/mínimo rapidamente”	Heap	`priority_queue`
“Inserir/remover no início E no final”	Fila dupla	`deque`

---

7. Exemplos práticos de melhoria

Padrão: Iteração e Acumulação

Muitos dos seus problemas seguem o padrão: “Leia N valores, processe algo sobre eles”.

Seu estilo antigo (Lista 01):

```
int n, i;
scanf("%d", &n);
int vet[n];
for (i = 0; i < n; i++) scanf("%d", &vet[i]);
```

Estilo competitivo C++ (Lista 03 em diante):

```
int n;
cin >> n;
vector<int> v(n);
for (int &x : v) cin >> x;  // Range-based for com referência
```

Padrão: Encontrar máximo/mínimo

Seu estilo (2854 — Lista 02):

```
int maior;
for (i = 0; i < n; i++) {
    scanf("%d", &vet[i]);
    if (i == 0) maior = vet[i];
    else if (maior < vet[i]) maior = vet[i];
}
```

Estilo com STL:

```
vector<int> v(n);
for (int &x : v) cin >> x;
int maior = *max_element(v.begin(), v.end());
```

Ou ainda mais simples, sem guardar o vetor:

```
int maior = INT_MIN;
for (int i = 0; i < n; i++) {
    int x; cin >> x;
    maior = max(maior, x);
}
```

Padrão: Multiple Test Cases com EOF

Seu estilo (correto, mas pode ser mais limpo):

```
while (scanf("%d", &n) != EOF) { ... }
```

Estilo C++ (idêntico em comportamento):

```
while (cin >> n) { ... }
```

Padrão: Multiple Test Cases com sentinela (n = 0)

```
// Você já usa corretamente:
cin >> n;
while (n != 0) {
    // ...
    cin >> n;
}

// Forma mais compacta:
while (cin >> n && n != 0) { ... }
// Ou simplesmente:
while (cin >> n, n) { ... }
```

---

8. Complexidade e análise de algoritmos

Tabela de referência: Limites seguros

N (tamanho da entrada)	Complexidade máxima aceitável	Exemplo de algoritmo
$N \leq 20$	$O(2^N)$ ou $O(N!)$	Backtracking, força bruta completa
$N \leq 1.000$	$O(N^2)$	Bubble Sort, 2 loops aninhados
$N \leq 100.000$	$O(N \log N)$	`sort()`, Merge Sort
$N \leq 1.000.000$	$O(N)$	Prefix Sum, contagem linear
$N \leq 10^9$	$O(\log N)$ ou $O(1)$	Busca binária, fórmula matemática

Como estimar: A “regra dos 10^8”

Em competição, considere que o computador executa aproximadamente 10^8 operações simples por segundo.

Se $N = 10^5$ e seu algoritmo é $O(N^2)$:

$$10^5 \times 10^5 = 10^{10} \gg 10^8 \implies \text{TLE (Time Limit Exceeded)}$$

Se $N = 10^5$ e seu algoritmo é $O(N \log N)$:

$$10^5 \times 17 \approx 1{,}7 \times 10^6 \ll 10^8 \Longrightarrow OK!$$

Complexidades dos seus códigos

Problema	Seu algoritmo	Complexidade	Poderia ser
1192 (ordenação)	Bubble Sort	$O(N^2)$	$O(N \log N)$ com `sort()`
1457 (ordenação)	Bubble Sort	$O(N^2)$	$O(N \log N)$ com `sort()`
1985 (soma intervalo)	Loop por query	$O(Q \times N)$	$O(N + Q)$ com Prefix Sum
1142 (fila com shift)	Shift manual	$O(Q \times N)$	$O(Q)$ com deque

---

9. Grafos

9.1 Conceito Fundamental

Um **grafo** $G = (V, E)$ é um conjunto de **vértices** (nós) V e **arestas** (ligações) E que conectam pares de vértices.

Exemplo do dia a dia: - Cidades = vértices, estradas = arestas. - Pessoas em uma rede social = vértices, amizades = arestas.

9.2 Tipos de Grafos

Tipo	Descrição	Exemplo
Não-direcionado	Arestas de mão dupla	Amizade (se A é amigo de B, B é amigo de A)
Direcionado (Dígrafo)	Arestas de mão única	Twitter (A segue B, mas B não necessariamente segue A)
Ponderado	Arestas com peso/custo	Mapa com distâncias entre cidades
Não-ponderado	Arestas sem peso	Labirinto (cada passo custa 1)

9.3 Representação em C++

OPÇÃO 1: LISTA DE ADJACÊNCIA (USE EM 95% DOS PROBLEMAS)

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 100005;
vector<int> adj[MAXN]; // Array de vectors

int main() {
    int n, m;
    cin >> n >> m;

    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v); // u → v
        adj[v].push_back(u); // v → u (se não-direcionado)
    }
}
```

Memória: $O(V + E)$ — eficiente. **Achar vizinhos:** $O(grau(v))$ — rápido. **Usar quando:** V pode ser grande ($\leq 10^5$). Ou seja, **quase sempre**.

OPÇÃO 2: MATRIZ DE ADJACÊNCIA (APENAS SE $V \leq 1000$)

```
int adj[1001][1001]; // 0 = sem aresta, 1 = com aresta

// Leitura:
adj[u][v] = 1;
adj[v][u] = 1; // se não-direcionado
```

Memória: $O(V^2)$ — ruim para V grande. **Consultar "u e v estão conectados?":** $O(1)$ — instantâneo. **Usar quando:** V é pequeno (≤ 1000) e você precisa consultar conexões diretas com frequência.

COM PESOS (GRAFO PONDERADO):

```
// Lista de adjacência ponderada:
vector<pair<int, int>> adj[MAXN]; // {vizinho, peso}
adj[u].push_back({v, w});
adj[v].push_back({u, w});
```

9.4 DFS — Busca em Profundidade

O QUE É?

Algoritmo que explora um grafo indo **o mais fundo possível** antes de voltar. Usa recursão (ou pilha explícita).

ANALOGIA:

Imagine um labirinto. A DFS é: "Vá sempre em frente até bater na parede. Quando bater, volte até a última bifurcação e tente outro caminho."

IMPLEMENTAÇÃO:

```
bool visitado[MAXN];

void dfs(int u) {
    visitado[u] = true;
    // Processar o nó u aqui, se necessário

    for (int v : adj[u]) {
        if (!visitado[v]) {
            dfs(v);
        }
    }
}
```

APLICAÇÕES:

- **Contar componentes conexas:** Quantos "grupos isolados" existem no grafo.
- **Verificar se dois nós estão conectados.**
- **Detectar ciclos.**
- **Flood Fill** (preencher uma região em uma matriz).

EXEMPLO: CONTAR COMPONENTES CONEXAS

```
int componentes = 0;
memset(visitado, false, sizeof(visitado));

for (int i = 1; i <= n; i++) {
    if (!visitado[i]) {
        dfs(i);
        componentes++;
    }
}
cout << componentes << "\n";
```

Complexidade: $O(V + E)$ — visita cada vértice e cada aresta uma única vez.

9.5 BFS — Busca em Largura

O QUE É?

Algoritmo que explora o grafo **camada por camada**, usando uma **fila (queue)**. Primeiro visita todos os vizinhos diretos, depois os vizinhos dos vizinhos, etc.

ANALOGIA:

Jogue uma pedra na água. A onda se espalha em círculos concêntricos. A BFS explora o grafo exatamente assim.

IMPLEMENTAÇÃO:

```
int dist[MAXN]; // Distância do nó de origem até cada nó

void bfs(int origem) {
    memset(dist, -1, sizeof(dist)); // -1 = não visitado
    queue<int> q;

    dist[origem] = 0;
    q.push(origem);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] == -1) { // Não visitado
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}
```

APLICAÇÕES:

- **Menor caminho em grafos não-ponderados** (cada aresta tem custo 1).
- “Quantos passos mínimos para ir de A até B?”
- Labirinto: menor número de movimentos para sair.

Complexidade: $O(V + E)$.

9.6 DFS vs BFS — Quando usar qual?

Característica	DFS	BFS
Estrutura auxiliar	Pilha (recursão)	Fila (`queue`)
Exploração	Vai fundo primeiro	Vai largo primeiro
Menor caminho (sem peso)	**Não garante**	**Garante** ✓
Componentes conexas	✓ Funciona	✓ Funciona
Detecção de ciclos	✓ Fácil	Possível, mas mais complexo
Flood Fill em matriz	✓ Natural	✓ Natural
Memória (grafos profundos)	Pode estourar a stack	Geralmente mais seguro

> **Regra prática:** - Precisa de **menor caminho** (sem peso)? → **BFS**. - Precisa de **componentes conexas** ou **ciclos**? → **DFS** (mais simples de implementar).

9.7 Exemplo Progressivo: Problema de Grafo

Enunciado típico de simulado: > Em uma rede com N computadores e M cabos de rede, cada cabo conecta dois computadores. Quantos grupos isolados de computadores existem? (Dois computadores estão no mesmo grupo se há algum caminho entre eles.)

Tradução: Contar **componentes conexas** de um grafo não-direcionado.

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 100005;
vector<int> adj[MAXN];
bool vis[MAXN];

void dfs(int u) {
    vis[u] = true;
    for (int v : adj[u])
        if (!vis[v]) dfs(v);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n, m;
    cin >> n >> m;

    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    int comp = 0;
    for (int i = 1; i <= n; i++) {
        if (!vis[i]) {
            dfs(i);
            comp++;
        }
    }

    cout << comp << "\n";
    return 0;
}
```

9.8 Problema do tipo “Labirinto” (BFS em Matriz)

Enunciado típico: > Uma matriz $N \times M$ representa um mapa. ‘.’ é caminho livre, ‘#’ é parede. Encontre o menor número de passos de *(r_1, c_1)* até *(r_2, c_2)*.

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m;
char grid[1001][1001];
int dist[1001][1001];

int dx[] = {0, 0, 1, -1}; // Direções: direita, esquerda, baixo, cima
int dy[] = {1, -1, 0, 0};

int bfs(int r1, int c1, int r2, int c2) {
    memset(dist, -1, sizeof(dist));
    queue<pair<int, int>> q;

    dist[r1][c1] = 0;
    q.push({r1, c1});

    while (!q.empty()) {
        auto [r, c] = q.front(); // C++17: structured binding
        q.pop();

        if (r == r2 && c == c2) return dist[r][c];

        for (int d = 0; d < 4; d++) {
            int nr = r + dx[d];
            int nc = c + dy[d];

            if (nr >= 0 && nr < n && nc >= 0 && nc < m
                && grid[nr][nc] != '#' && dist[nr][nc] == -1) {
                dist[nr][nc] = dist[r][c] + 1;
                q.push({nr, nc});
            }
        }
    }

    return -1; // Sem caminho
}
```

> **Reconhecimento:** Se o problema mostra uma **matriz/grid** e pede **menor caminho → BFS em matriz** com vetor de direções `dx[]` / `dy[]`.

---

10. STL aplicada a Grafos

STL	Uso em Grafos
`vector<int> adj[N]`	Lista de adjacência (padrão ouro)
`vector<pair<int,int>> adj[N]`	Lista de adjacência com pesos
`queue<int>`	BFS
`stack<int>`	DFS iterativa (alternativa à recursão)
`priority_queue<pair<int,int>>`	Dijkstra (menor caminho com pesos)
`set<int>`	Vizinhos únicos (grafos com arestas duplicadas)
`map<string, int>`	Converter nomes para IDs numéricos
`bool vis[N]` / `vector<bool> vis(n)`	Marcar nós visitados
`int dist[N]` / `vector<int> dist(n, -1)`	Distâncias na BFS

Exemplo: Converter nomes para IDs com `map`

Problema 2854 da sua Lista 02 (Árvore Genealógica) usa nomes como vértices:

```
map<string, int> id;
int proxId = 0;

int getId(string nome) {
    if (id.find(nome) == id.end()) {
        id[nome] = proxId++;
    }
    return id[nome];
}

// Agora você pode usar adj[getId("Maria")].push_back(getId("Joao"));
```

---

11. Checklist para os simulados

Antes de começar a codar

] Li o enunciado **duas vezes**?
] Identifiquei a **entrada** (formato, limites de N)?
] Identifiquei a **saída** (formato exato, espaços, quebras de linha)?
] Qual é a **complexidade máxima aceitável** dado N?
] Esse problema se encaixa em algum **padrão conhecido**?

Template de início

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    // Seu código aqui

    return 0;
}
```

Antes de submeter

] Testei com o **exemplo do enunciado**?
] Testei com **casos extremos** (N = 0, N = 1, valores negativos, empates)?
] Verifico se estou usando `=` e não `==` por acidente?
] O tipo é grande o suficiente? (`int` vs `long long`)
] Se usei `stack` / `queue`, verifiquei `empty()` antes de acessar?
] Se o grafo é não-direcionado, adicionei **ambas** as direções?
] Usei `"\n"` em vez de `endl` (performance)?

Erros que já cometi

Erro	Onde cometi	Consequência
`a[i] == 1` em vez de `a[i] = 1`	1301.cpp	Wrong Answer silencioso
`int vet[n]` com n não inicializado	1030.cpp (L2)	Comportamento indefinido
Soma de `D²` com `R` (dimensões erradas)	Exercício dos círculos	Wrong Answer
Faltou `using namespace std`	ex1.cpp (estudos)	Erro de compilação
`map<string,int> notas(n)`	ex1.cpp (estudos)	Map não aceita tamanho inicial
`sort()` no `map`	ex1.cpp (estudos)	Map não permite `sort`

---

12. Exercícios de revisão

Estes exercícios cobrem os padrões que você precisa consolidar. Tente resolver **sem olhar a solução**.

FÁCIL R1 — Frequência de Letras

> Dada uma string S, imprima cada letra que aparece e quantas vezes ela aparece, em ordem alfabética. Ignore maiúsculas/minúsculas (trate tudo como minúsculo).
>
> Entrada: `AaBbCcAa` → Saída: `a 4`, `b 2`, `c 2`

Dica: Qual STL organiza automaticamente em ordem e conta frequência?

MÉDIO R2 — K Maiores Elementos

> Dados N inteiros e um valor K, imprima os K maiores valores em ordem decrescente. ($N \leq 10^5$)

Dica: Qual é a complexidade de ordenar e pegar os K últimos?

MÉDIO R3 — Ilha no Mapa

> Uma matriz $N \times M$ contém `.` (água) e `#` (terra). Ilhas são grupos de `#` conectados (horizontal/verticalmente). Quantas ilhas existem?

Dica: Cada ilha é uma... componente conexa! Qual algoritmo conta componentes conexas?

DIFÍCIL R4 — Menor Caminho no Labirinto

> Uma matriz $N \times M$ contém `.` (caminho livre), `#` (parede), `S` (início) e `E` (fim). Qual o menor número de passos de `S` até `E`? Se não há caminho, imprima `-1`.

Dica: Menor caminho sem peso → qual algoritmo?

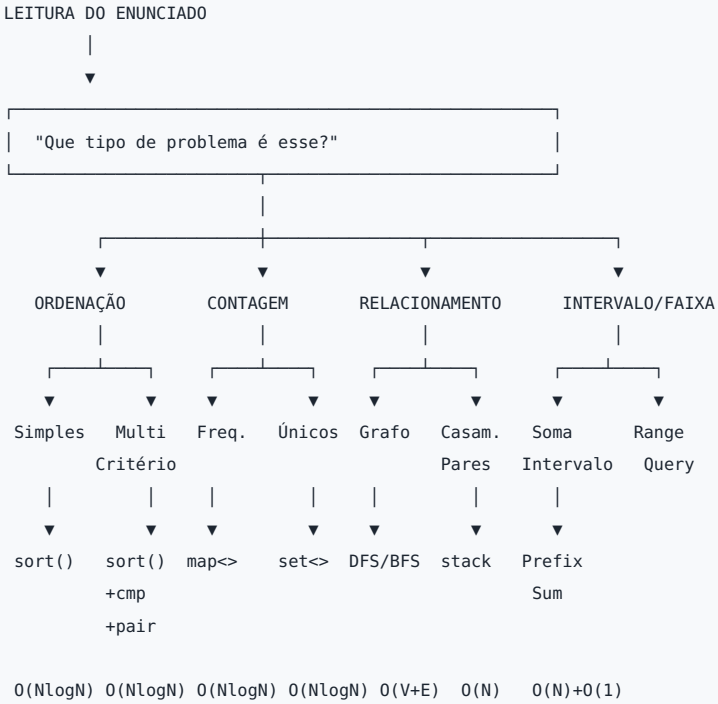
DIFÍCIL R5 — Prefix Sum + Query

> Dado um vetor de N inteiros e Q queries, cada query pede a soma dos elementos no intervalo $[L, R]$. ($N, Q \leq 10^5$)

Dica: Qual técnica faz cada query em $O(1)$ após um pré-processamento de $O(N)$?

---

13. Mapa Mental de Reconhecimento



Resumo em frases:

Padrão	→ Pense em...
"Ordenar"	`sort()`
"Ordenar por mais de 1 critério"	`sort() + pair ou cmp`
"Contar frequência"	`map<chave, int>`
"Elementos únicos/distintos"	`set<>`
"Inserir/remover no final"	`vector` ou `stack`
"Inserir/remover em ambas extremidades"	`deque`
"FIFO / ordem de chegada"	`queue`
"LIFO / desfazer / parênteses"	`stack`
"Soma de intervalo (múltiplas queries)"	**Prefix Sum**
"Relação entre pares de coisas"	**Grafos**
"Menor caminho sem peso"	**BFS**
"Componentes conexas / grupos"	**DFS**
"Menor caminho com peso"	**Dijkstra** (pq + BFS)
"Chave → Valor"	`map<>`
"Maior/menor elemento dinamicamente"	`priority_queue`

---

14. Próximos pontos a estudar

Prioridade Alta (essenciais para simulado)

1. **Prefix Sum** — Você já enfrentou esse padrão (1985) e resolveu com loop. Precisa dominar a técnica.
2. **DFS e BFS na prática** — Fazer pelo menos 5 problemas de grafos (componentes conexas, labirinto, flood fill).
3. **Busca Binária** — Técnica fundamental que ainda não apareceu nos seus códigos.
4. **Ordenação customizada avançada** — Treinar `sort()` com funções de comparação para 2-3 critérios.

Prioridade Média (segundo nível)

5. **Programação Dinâmica (DP)** — O próximo grande salto conceitual.
6. **Dijkstra** — Menor caminho com pesos.
7. **Union-Find** — Alternativa eficiente para componentes conexas.
8. **Árvores** — Conceito de raiz, filhos, ancestrais.

Prioridade Futura

9. **Segment Tree** — Queries de intervalo avançadas.
10. **Programação Gulosa (Greedy)** — Estratégias de otimização local.
11. **Teoria dos Números** — MDC, MMC, primalidade, crivo de Eratóstenes.

> **IMPORTANTE**
>
> **Mensagem do treinador:** Você evoluiu de “C puro com Bubble Sort” para “C++ com STL real” em poucas semanas. Isso é significativo. Agora o próximo passo não é aprender mais estruturas — é **automatizar o reconhecimento**. Quando você bater o olho num problema e pensar “isso é um map” ou “isso é BFS” em 30 segundos, você estará pronto para competir de verdade.